

API de verificación

El contrato JSON entre Yalqui, Truora y las centrales de riesgo — y un bloqueador de red que hay que resolver antes de escribir la primera línea.

Complementa el modelo de datos, módulo M4 Alcance screening del inquilino Estado propuesta

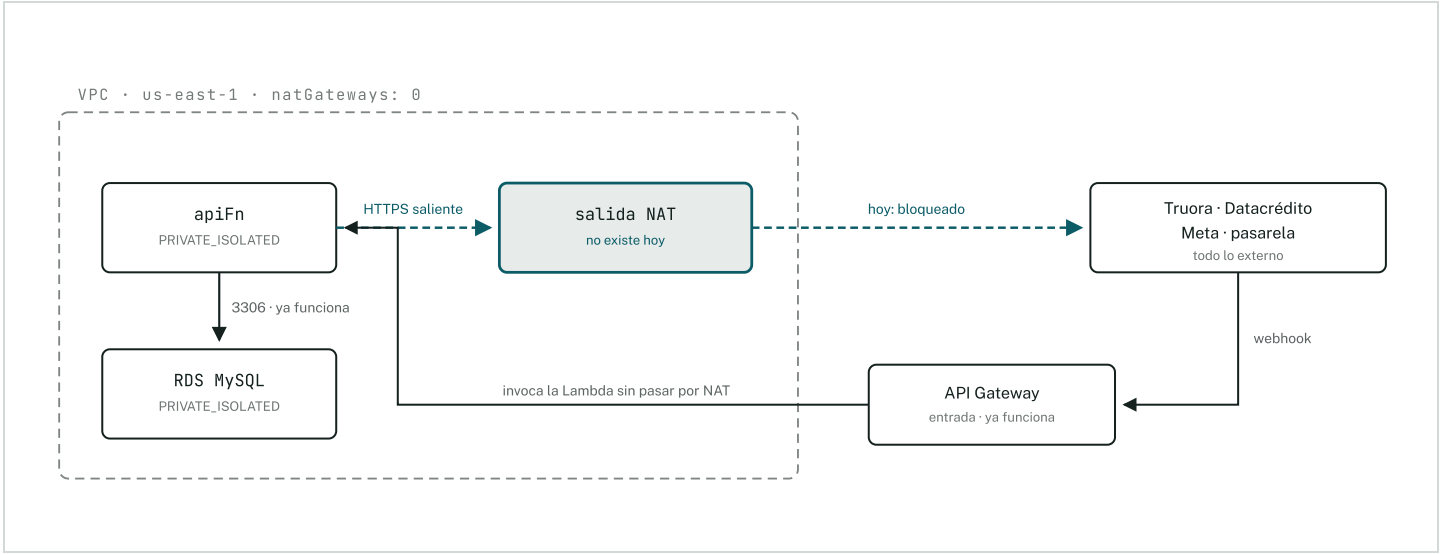
00 El bloqueador de red	01 Capa anticorrupción	02 Superficie interna
03 Resultado normalizado	04 Webhooks	05 Máquina de estados
06 Errores y su lectura	07 Mapeo por proveedor	08 Seguridad y datos
09 Qué falta confirmar		

00 Antes del contrato: no hay ruta de salida

Verifiqué la infraestructura desplegada antes de diseñar nada. `infra/lib/infra-stack.ts:42` crea la VPC con `natGateways: 0`, y las tres Lambdas corren en subredes `PRIVATE_ISOLATED`. Una subred aislada no tiene ruta a internet: ni NAT, ni gateway de internet. El único endpoint configurado es el de S3.

La Lambda del API no puede llamar a Truora. Tampoco a Datacrédito, ni a la API de WhatsApp, ni a una pasarela de link de pago, ni a un servicio de correo. La llamada no falla rápido: se queda colgada hasta que el `timeout` de 20 segundos la mata, que es la forma más confusa posible de descubrirlo.

Esto no es un problema de Truora. Es el mismo muro para **todas** las integraciones del diseño, así que conviene resolverlo en la capa de red una vez, y no integración por integración.



Solo falta la mitad de salida. Los webhooks entrantes ya funcionan: llegan por API Gateway, que invoca la Lambda desde fuera de la VPC y no necesita ruta de red. Lo que no existe es el camino inverso — que la Lambda inicie una llamada hacia afuera. Es una asimetría fácil de pasar por alto y explica por qué el hello actual funciona perfecto y la primera llamada a Truora se va a quedar colgada.

Tres formas de resolverlo

OPCIÓN	COSTO MENSUAL APROX.	QUÉ IMPLICA
NAT Gateway	~US\$32 + tráfico	Una línea en el CDK y funciona todo. Triplica la factura actual de ~US\$14, que es mucho para el volumen de arranque
Instancia NAT t4g.nano	~US\$7	Una EC2 pequeña haciendo de NAT. Hay que parchearla y es punto único de falla, pero para este tamaño la relación costo-beneficio es la mejor. Recomendada para arrancar
Worker fuera de la VPC + SQS	~US\$7	Una Lambda sin VPC hace las llamadas y devuelve por SQS a otra Lambda dentro. Serverless puro, pero el envío a SQS desde la VPC necesita su propio endpoint de interfaz y hay que repetir el patrón por cada integración

RECOMENDACIÓN Instancia NAT ahora, NAT Gateway cuando el volumen lo justifique. La tercera opción es la más elegante en papel y la que más piezas agrega: como el diseño tiene cinco integraciones salientes distintas, resolver la salida una vez en la red sale más barato que resolverla cinco veces en la aplicación. **LOS PRECIOS SON DE REFERENCIA Y HAY QUE VERIFICARLOS.**

Efecto colateral en la documentación: DEPLOYMENT.md dice hoy «Sin NAT Gateway — la Lambda solo necesita hablar con MySQL dentro de la VPC». Era cierto cuando se escribió y deja de serlo con este diseño. Conviene actualizarlo en el mismo cambio, o alguien va a volver a quitar el NAT por ahorrar.

01 Yalqui define su vocabulario, los proveedores se traducen

Truora y una central de riesgo no hablan el mismo idioma, ni entre ellas ni con el modelo. Truora responde sobre listas y procesos; la central responde sobre obligaciones y mora. Si esos formatos entran crudos al esquema, cambiar de proveedor obliga a reescribir la aplicación entera.

Por eso hay tres capas y no dos: **el contrato interno** que Yalqui define y no cambia, **un adaptador por proveedor** que traduce, y **el volcado crudo** que se guarda por si hay que auditar o reprocesar. Es exactamente lo que ya soportan las columnas hallazgos y respuesta de la tabla verificaciones.

CAPA	QUIÉN LA DEFINE	DÓNDE VIVE	CAMBIA CUANDO
Contrato interno	Yalqui	El tipo de hallazgos	Casi nunca. Es la promesa de la que dependen la app y la decisión del propietario
Adaptador	Yalqui, uno por proveedor	backend/src/verificacion/adaptadores/	Cuando el proveedor cambia su API. Es el único punto que se toca
Volcado crudo	El proveedor	verificaciones.respuesta	Nunca se interpreta en tiempo real. Existe para auditar y reprocesar

02 Superficie interna

El backend ya usa tRPC, así que lo que el frontend consume son procedimientos, no rutas REST. Cuatro alcanzan. El webhook sí necesita una ruta HTTP aparte, porque el proveedor no habla tRPC.

PROCEDIMIENTO	TIPO	QUÉ HACE
verificaciones.solicitar	mutation	Pide las verificaciones que falten, reutiliza las vigentes y devuelve el costo antes de gastarlo
verificaciones.estado	query	Consulta una o varias por id. Es lo que la pantalla consulta mientras espera
verificaciones.deAplicacion	query	Devuelve el paquete que respalda una aplicación, con el resumen que ve el propietario
verificaciones.reintentar	mutation	Reintenta solo las fallidas. No reintenta las rechazadas: eso no es un error
POST /webhooks/verificacion/:proveedor	ruta HTTP	Recibe el resultado asíncrono. Fuera de tRPC, con validación de firma

Solicitar

ENTRADA

```
// verificaciones.solicitar
{
  "sujeto": {
    "tipo": "inquilino",    // | "garante"
    "usuarioId": 4821
  },
  "tipos": [
    "identidad",
    "antecedentes_judiciales",
    "listas_restrictivas",
    "centrales_riesgo"
  ],
  "aplicacionId": 1907,
  "forzarNueva": false
}
```

SALIDA

```
{
  "solicitadas": [{
    "verificacionId": 55012,
    "tipo": "identidad",
    "estado": "en_proceso",
    "proveedor": "truora",
    "costo": 2500
  }],
  "reutilizadas": [{
    "verificacionId": 54120,
    "tipo": "centrales_riesgo",
    "resultado": "aprobado",
    "vigenteHasta": "2026-09-30"
  }],
  "bloqueadas": [{
    "tipo": "antecedentes_judiciales",
    "motivo": "consentimiento_faltante",
    "requiere": "consulta_antecedentes"
  }],
  "costoTotal": 2500
}
```

Las tres listas de la respuesta son el diseño, no un detalle. reutilizadas es lo que hace que aplicar a cinco inmuebles no cueste cinco consultas. bloqueadas separa «no se pudo» de «salió mal»: falta un consentimiento, no falló nada. Y costoTotal se devuelve **antes** de cobrarle al propietario, para que la interfaz pueda decir «esto cuesta \$2.500, ¿seguimos?» en vez de presentarle la factura después.

03 El resultado normalizado

Es el contrato que no cambia aunque cambien los proveedores. Se guarda en `verificaciones.hallazgos` y es lo único que la aplicación lee para decidir qué mostrarle al propietario.

```

{
  "version": "1.0",
  "tipo": "antecedentes_judiciales",
  "resultado": "con_observaciones",
  "resumen": "1 proceso civil vigente. Sin antecedentes penales ni listas restrictivas.",
  "hallazgos": [{
    "categoria": "proceso_judicial",
    "gravedad": "media",          // leve | media | grave
    "fuente": "rama_judicial",
    "descripcion": "Proceso ejecutivo, juzgado 12 civil de Bogotá",
    "fecha": "2024-03-11",
    "vigente": true
  }],
  "fuentesConsultadas": ["rama_judicial", "procuraduria", "contraloria", "policia", "ofac"],
  "fuentesFallidas": [],        // ← si trae algo, el resultado es parcial
  "consultadoAt": "2026-08-27T14:12:09Z"
}

```

COMPORTAMIENTO CREDITICIO · VÍA CENTRAL DE RIESGO

```

{
  "version": "1.0",
  "tipo": "centrales_riesgo",
  "resultado": "aprobado",
  "puntaje": 712,
  "escala": { "min": 150, "max": 950, "proveedor": "datacredito" },
  "resumen": "Sin obligaciones en mora. 4 obligaciones vigentes al día.",
  "endeudamientoTotal": 12000000,
  "cuotaMensualEstimada": 850000,  // ← el dato que de verdad sirve
  "hallazgos": [{
    "categoria": "obligacion_al_dia",
    "entidad": "Banco X",
    "saldo": 12000000,
    "moraMaximaMeses": 0
  }],
  "fuentesConsultadas": ["datacredito"],
  "fuentesFallidas": [],
  "consultadoAt": "2026-08-27T14:12:40Z"
}

```

Tres decisiones dentro de este JSON

01 fuentes Fallidas no es opcional

Si la Procuraduría no respondió, un resultado «limpio» no está limpio: está incompleto. Sin esta lista, una caída de una fuente se le muestra al propietario como si la persona no tuviera nada, que es la falla silenciosa más peligrosa de todo el diseño. Cuando trae elementos, la interfaz debe decir «verificación parcial», no «aprobado».

02 El puntaje viaja con su escala

712 no significa nada solo. Cada central usa su propio rango y cambiarlo de proveedor cambiaría el número sin cambiar a la persona. Guardar escala junto al puntaje es el principio de transparencia del modelo aplicado aquí: el propietario ve «712 de 950», no un número suelto que va a interpretar mal.

03 cuota Mensual Estimada vale más que el puntaje

Para decidir un arriendo, la pregunta útil no es «¿qué puntaje tiene?» sino «¿le alcanza?». Con la cuota estimada, los ingresos declarados en `perfiles_inquilino` y el canon del inmueble, la app puede mostrar una relación de capacidad de pago — que es información accionable, a diferencia de un score crediticio que casi nadie sabe leer.

04 Webhooks

Los *checks* de antecedentes son asíncronos: se pide, devuelve un identificador, y el resultado llega minutos después. Esa entrada ya funciona sin tocar la red, porque API Gateway invoca la Lambda desde afuera.

LO QUE YALQUI PRODUCE INTERNAMENTE TRAS NORMALIZAR

```
{
  "proveedor": "truora",
  "evento": "verificacion.completada",
  "idExterno": "chk_01J8ZQ4M...",
  "idEvento": "evt_01J8ZQ7P...", // para deduplicar reenvíos
  "ocurridoAt": "2026-08-27T14:12:09Z",
  "firmaValida": true,
  "hallazgos": { /* el contrato normalizado de la sección 03 */ }
}
```

Cinco reglas del receptor

01 Validar la firma antes de parsear

El cuerpo llega de internet abierto. Se verifica el HMAC contra el secreto guardado en Secrets Manager — el mismo patrón que ya usan las credenciales de base de datos y el JWT — y solo después se convierte a objeto. Un cuerpo sin firma válida se descarta con 401 sin registrar su contenido.

02 Responder rápido, procesar después

El proveedor espera un 200 en pocos segundos o reintenta. El receptor guarda el evento y contesta; la normalización y la escritura van aparte. Un webhook que hace todo el trabajo antes de responder termina generando reintentos que duplican el trabajo.

03 No confiar en el contenido para el resultado final

El webhook dice «ya está lista», y Yalqui vuelve a consultar el resultado por API antes de darlo por bueno. Es una llamada extra a cambio de que un webhook falsificado no pueda aprobar a nadie. En una decisión que define si alguien consigue vivienda, la llamada extra sobra poco.

04 Deduplicar por `idEvento`, no por `idExterno`

Un mismo *check* puede generar varios eventos legítimos — iniciado, completado, corregido. Deduplicar por el id del *check* se comería el segundo evento válido. Por eso la unicidad va sobre el par proveedor + id de evento.

05 Un evento que no corresponde a nada se guarda igual

Si llega un webhook cuyo `idExterno` no está en la base, algo se desincronizó y hay que poder verlo. Se registra como huérfano y se alerta, en vez de descartarlo en silencio.

Una tabla más para el modelo: `eventos_webhook` — `id`, `proveedor`, `id_evento_externo`, `tipo`, `cuerpo_JSON`, `firma_valida`, `procesado_at`, `error`, `created_at`, con `UNIQUE(proveedor, id_evento_externo)`. No es solo para Truora: WhatsApp confirma entregas por webhook, la pasarela de link de pago confirma pagos, y las centrales notifican actualizaciones. Una sola tabla genérica para los cuatro casos, que además permite reprocesar sin pedirle al proveedor que reenvíe. Sube el modelo a **65 tablas**, en fase 2.

05 Máquina de estados

ESTADO	SE ENTRA CUANDO	SE SALE A	QUÉ VE EL PROPIETARIO
solicitada	Se creó la fila, aún no se llamó al proveedor	en_proceso · fallida	«Preparando»
en_proceso	El proveedor aceptó y devolvió id_externo	completada · fallida	«Consultando, suele tardar unos minutos»
completada	Llegó el resultado y se normalizó	expirada, al pasar vigente_hasta	El resultado: aprobado, con observaciones o rechazado
fallida	Error técnico, caída o vencimiento del plazo	en_proceso, al reintentar	«No pudimos verificar» + botón de reintentar. Nunca «rechazado»
expirada	Pasó la vigencia del proveedor	—	«Vencida, hay que repetirla». Se conserva para auditoría

PLAZOS Si no llega webhook a los 5 minutos, se consulta por API. Si a los 30 minutos sigue sin resolverse, pasa a fallida con motivo timeout_proveedor y no se le cobra al propietario. Reintentar reusa el mismo id_externo cuando el proveedor lo permite, para no pagar dos veces la misma consulta.

06 Errores, y qué significa cada uno

Esta tabla es la parte del diseño con más consecuencia sobre una persona real. Un error de infraestructura y un antecedente encontrado no son lo mismo, y mostrarlos igual hace que alguien se quede sin vivienda porque una API estuvo caída.

CÓDIGO	QUÉ PASÓ	SE COBRA	REINTENTA	QUÉ VE EL PROPIETARIO
consentimiento_faltante	No hay autorización vigente. Ni se llamó al proveedor	No	Al autorizar	«Falta que el inquilino autorice esta consulta»
datos_invalidos	Cédula mal digitada o incompleta	No	Al corregir	«Revisá el número de documento»
sujeto_no_encontrado	El proveedor respondió: no hay registro de esa persona	Sí	No	«Sin información disponible». No es un rechazo
proveedor_no_disponible	Caída, 5xx o timeout	No	Automático	«No pudimos verificar ahora, reintentando»
resultado_parcial	Respondió, pero con fuentesFallidas	Sí	Manual	«Verificación incompleta» + qué fuente faltó
credenciales_invalidas	La llave de Yalqui venció o se revocó	No	No	Nada. Es problema de Yalqui: alerta interna
cuota_excedida	Se agotó el cupo contratado con el proveedor	No	Diferido	«En cola». Alerta interna urgente

La regla que ninguna de estas filas puede romper: un fallo técnico nunca se convierte en resultado = rechazado. estado y resultado son columnas distintas justamente por esto — el estado dice si la consulta funcionó, el resultado dice qué encontró. Colapsarlas en una sola sería más simple de programar y significaría que una caída de Truora, para efectos del propietario, es indistinguible de un antecedente penal.

07 Mapeo por proveedor

Cada adaptador implementa la misma interfaz y traduce en las dos direcciones. Esta tabla es el enrutamiento que vive como datos en `proveedores_verificacion.tipos_soportados`, no como `if` en el código.

TIPO EN YALQUI	PROVEEDOR	QUÉ DEVUELVE	VIGENCIA SUGERIDA
identidad	Truora	Validación de documento y coincidencia biométrica	12 meses
documento	Truora	Autenticidad del documento y prueba de vida	12 meses
antecedentes_judiciales	Truora	Procesos en Rama Judicial	3 meses
antecedentes_policiales	Truora	Policía, Procuraduría, Contraloría	3 meses
listas_restrictivas	Truora	OFAC y listas de control	3 meses
centrales_riesgo	Central	Puntaje, obligaciones, mora, endeudamiento	1 mes
laboral	Manual	Certificado revisado por una persona	3 meses

CONVIVENCIA OBLIGADA Truora cubre identidad y antecedentes, pero no reporte crediticio; la central cubre lo crediticio y no lo demás. No hay un proveedor que reemplace a los dos, así que el adaptador no es una precaución teórica: desde el primer día hay que hablar dos idiomas. Las vigencias son propuestas y van en `proveedores_verificacion.vigencia_dias` — la crediticia es corta a propósito, porque es la que más rápido envejece.

Traducción del resultado

Cada proveedor tiene su forma de decir «pasó». La conversión al resultado de Yalqui es una regla configurable por proveedor y no una constante en el código, porque los umbrales se van a ajustar con datos reales.

RESULTADO YALQUI	DESDE ANTECEDENTES	DESDE CENTRAL DE RIESGO
aprobado	Sin hallazgos, y fuentes fallidas vacío	Puntaje sobre el umbral y sin mora vigente
con_observaciones	Hallazgos de gravedad leve o media	Puntaje en zona intermedia, o mora saldada
rechazado	Hallazgo grave o lista restrictiva	Mora vigente sobre el umbral

`OJO CON CON_OBSERVACIONES` No es un estado intermedio decorativo: es el que obliga a que decida una persona y no una regla. Un proceso civil de 2019 ya resuelto y una mora vigente de tres meses caen los dos ahí, y significan cosas distintas. El sistema muestra el hallazgo con su fecha y su gravedad; la decisión la firma el propietario, y queda en `aplicaciones.decidida_por_id` con su motivo.

08 Seguridad y trato de los datos

Consentimiento como compuerta, no como registro

Consultar centrales de riesgo en Colombia exige autorización expresa y previa. El adaptador no confía en que la aplicación ya verificó: comprueba consentimientos justo antes de llamar y, si falta, devuelve `consentimiento_faltante` sin gastar un peso. Es la única forma de que la regla no se pierda en el primer refactor.

El volcado crudo caduca

`verificaciones.respuesta` contiene los datos más sensibles del sistema: procesos judiciales, obligaciones financieras, documentos. Se conserva 90 días para auditar y reprocesar, y después se purga dejando solo hallazgos, que es el resumen que la decisión necesita. Guardar todo para siempre es más fácil y convierte la base en un objetivo.

Nunca al log

Ni el cuerpo del webhook, ni la respuesta del proveedor, ni el número de documento completo. CloudWatch no tiene el mismo control de acceso que la base de datos y los logs se comparten con más gente de la que uno cree. Se registra el `id_externo` y el estado; eso alcanza para depurar.

El inquilino ve lo mismo que el propietario

Si una verificación arroja un hallazgo que le cuesta una aplicación, la persona tiene derecho a saber cuál fue y de qué fuente salió — es el mismo principio de transparencia del score. Y si el dato está mal, el reclamo va contra la fuente, no contra Yalqui: la interfaz debe decir de dónde salió para que sepa a dónde ir.

09 Qué falta confirmar antes de construir

El contrato interno de arriba lo definimos nosotros y es estable. Lo que no puedo dar por cierto es el detalle exacto del lado de los proveedores: nombres de campos, esquema de firma y modelo comercial cambian, y escribí esto sin acceso a sus documentos vigentes ni a un contrato firmado.

01 La superficie real de Truora

Nombres de endpoint, forma exacta del *payload*, esquema de firma del webhook y catálogo de fuentes por país. El diseño asume el patrón que usan: consulta asíncrona, identificador, webhook. Esa forma general es la que sostiene la máquina de estados; los nombres concretos se ajustan en el adaptador sin tocar nada más.

02Cuál central, y bajo qué contrato

Datacrédito y TransUnion tienen modelos comerciales distintos, y algunos exigen ser entidad vigilada o ir por un intermediario habilitado. Esto puede condicionar si Yalqui consulta directo o revende. Es la pregunta con más riesgo de bloquear la fase 2, y conviene resolverla temprano porque es comercial y legal, no técnica.

03Costo por consulta

Llena `proveedores_verificacion.costo_consulta` y define el precio del servicio Screening. Sin ese número, el margen del servicio es una suposición.

04Umbrales de aprobación

Desde qué puntaje es aprobado, desde qué mora es rechazado. Empezaría deliberadamente ancho — casi todo a `con_observaciones` — y lo iría cerrando con datos propios. Un umbral apretado desde el día uno rechaza gente que habría pagado bien, y eso no se ve en ninguna métrica.

ORDEN SUGERIDO

Primero la salida de red, porque sin eso nada de lo demás se puede probar ni siquiera en local contra el ambiente real. Después el adaptador de Truora, que es el que valida si el contrato de arriba aguanta. La central de riesgo al final, porque su bloqueador es comercial y no depende de nosotros.

Sigo sin escribir código. Cuando lo aprobés, esto se implementa junto con la fase 2 del [modelo de datos](#); la fase 1 no necesita ninguna llamada externa y puede avanzar en paralelo.